FitTracker — Especificación de Features (v1.0)

Contexto del usuario

- Objetivo principal: ganar fuerza y masa muscular usando máquinas
- Rutina: 5 días/semana (Lunes a Viernes), enfoque fuerza + postura
- Nivel: principiante en etapa inicial
- Plataforma: Android nativo (Kotlin)
- Interés secundario: Data Science / ML aplicado a los propios datos

---

Fase 1 — MVP (Core de entrenamiento)

1.1 Gestión de Rutinas

- Crear rutinas con nombre, días asignados y lista de ejercicios
- Cada ejercicio define: nombre, máquina asociada, series objetivo, rango de reps objetivo
- Rutina precargada: la rutina actual del usuario (Lunes a Viernes con ejercicios ya definidos)
- Posibilidad de duplicar o editar rutinas sin perder el historial previo

1.2 Registro de Sesión (pantalla principal de uso)

- Al iniciar sesión, la app carga la rutina del día automáticamente según el día de la semana
- Por cada ejercicio, el usuario registra: peso utilizado, series completadas, reps por serie
- Campo de notas por ejercicio (ej: "sentí dolor en hombro", "fácil, subir peso")
- Timer de descanso integrado configurable (default: 75 segundos), con vibración/sonido al terminar
- Para ejercicios de tiempo (planchas, bird dog): timer de conteo en lugar de reps
- Marcar sesión como completada al finalizar

1.3 RPE y Estado Diario

- Al terminar cada sesión: slider de RPE (1–10) con descripción breve (ej: "muy fácil" a "al límite")
- Registro opcional de: calidad de sueño (1–5) y nivel de energía (1–5)
- Estos datos alimentan análisis posteriores (correlación sueño-rendimiento)

1.4 Progresión de Fuerza

- Gráfico de evolución de peso por ejercicio a lo largo del tiempo (línea temporal)

- Vista de comparación: primera sesión vs. última sesión por ejercicio
- Indicador de racha: cuántas sesiones consecutivas sin faltar
- **Alerta de plateau**: notificación cuando un ejercicio lleva 3 sesiones sin progresar en peso o reps
- **Sugerencia de progresión**: si el usuario completó todas las reps del rango objetivo en 3 sesiones consecutivas, la app sugiere subir el peso un escalón

1.5 Historial

- Calendario visual con días entrenados marcados
- Vista de sesión pasada: todos los ejercicios con pesos y reps registrados
- Filtro por ejercicio: ver toda la historia de un ejercicio específico

---

Fase 2 — Métricas Corporales

2.1 Registro Corporal (semanal)

- Peso corporal (kg)
- Medidas de músculos clave: bícep, muslo, pantorrilla, cintura, pecho (cm)
- % de grasa corporal (entrada manual o calculada por fórmulas Navy/Jackson-Pollock)
- IMC calculado automáticamente en base a peso y altura del perfil
- Foto de progreso opcional (almacenada localmente)

2.2 Dashboard de Progreso Corporal

- Gráfico de evolución de peso corporal en el tiempo
- Gráfico por medida muscular
- Tendencia de % grasa
- Resumen semanal: qué mejoró, qué se estancó

Fase 3 — IA y Análisis Inteligente

3.1 Conexión con IA (API externa, ej: Claude o GPT)

- La IA recibe como contexto los datos reales del usuario:
    - Historial de pesos y reps de los últimos 30 días

    - RPE y estado físico reciente
    - Métricas corporales
    - Rutina actual
- Con ese contexto puede dar consejos específicos y relevantes, no genéricos
- Ejemplos de consultas útiles:
    - "¿Cuál es mi ejercicio con mejor progresión?"
    - "¿Estoy sobreentrenando algún grupo muscular?"
    - "¿Debería ajustar mi rutina dado mi progreso?"

3.2 Detección automática de patrones (ML local — futura integración Data Science)

- **Regresión lineal**: proyectar cuándo el usuario podría alcanzar un peso objetivo en un ejercicio
- **Clustering de sesiones**: identificar qué días/condiciones (sueño, energía) correlacionan con mejor rendimiento
- **Series de tiempo**: analizar tendencias de progresión por grupo muscular
- Nota: estos modelos se pueden desarrollar en Python primero (para aprender DS), y luego convertir a TensorFlow Lite para correr en el dispositivo, o consumir desde un backend Python (FastAPI)

Fase 4 — Alimentación (simplificada)

4.1 Registro de comidas rápido

- Templates de comidas frecuentes (guardadas por el usuario)
- Registro por macros manuales: proteína (g), carbohidratos (g), grasas (g), calorías
- Meta diaria de proteína calculada automáticamente (referencia: 1.6–2.2g por kg de peso corporal)
- Indicador visual diario de % cumplimiento de meta proteica

Stack Técnico Propuesto

Capa	Tecnología
Frontend	Android nativo — Kotlin
UI	Jetpack Compose
Navegación	Navigation Component

Capa	Tecnología
Base de datos local	Room (SQLite)
Gráficos	Vico o MPAndroidChart
Backend / IA	Python + FastAPI (para análisis ML y proxy de IA)
ML local futuro	TensorFlow Lite
API de IA	Anthropic Claude API o OpenAI

Estructura de Datos Principal (esquema conceptual)

```
Usuario
  └── perfil: nombre, altura, edad, sexo

Rutina
  └── días asignados
  └── Ejercicios (nombre, máquina, series_obj, reps_min, reps_max)

Sesión
  └── fecha, rutina_id, RPE, sueño, energía, completada
  └── SeriesEjercicio (ejercicio_id, peso_kg, reps, notas)

MétricaCorporal
  └── fecha, peso, % grasa, medidas por zona

ComidaDiaria (Fase 4)
  └── fecha, proteínas, carbos, grasas, calorías
```

Prioridad de Desarrollo Sugerida

1. Perfil de usuario + base de datos Room
2. Carga de rutina por día + pantalla de sesión activa
3. Timer de descanso
4. Historial y gráfico de progresión por ejercicio
5. RPE y estado diario

6. Alertas de plateau y sugerencias de progresión
7. Métricas corporales (Fase 2)
8. Integración IA con contexto real (Fase 3)
9. ML / análisis de patrones (Fase 3 avanzada)
10. Registro de alimentación (Fase 4)